Chapter 1

INTRODUCTION

The dominant paradigm for personal computing has remained largely unchanged since the early 1980s, when the graphical desktop — icons, menus, and pointer-driven interaction — was established through the Xerox Star and Apple Lisa lineage. Despite its longevity, a parallel research tradition has long explored whether users should instead be able to describe what they want in natural language and have the computer carry it out. The earliest serious system in this tradition, the Berkeley UNIX Consultant, was already operational by the late 1980s [1]. Decades later, the rise of large language models (LLMs) and multimodal foundation models[1] has reopened this thread with substantially greater technical capability.

The field that has emerged sits at the intersection of human-computer interaction, operating systems, natural language processing, and computer vision. On the shell side, recent work has demonstrated that LLMs can translate natural language into Bash commands with usable accuracy and can be embedded as a Unix primitive with appropriate safety guardrails [2, 3]. On the graphical side, so-called computer-using agents[2] now operate desktop and mobile applications by combining native APIs with vision-based grounding [16, 18]. On the web, language-guided agents complete tasks across realistic websites with measurable but limited reliability [8, 9]. A converging research thread approaches the same technical problem from an accessibility perspective: speech-driven GUI command mapping for users with motor and speech impairments has progressed from voice-to-text dictation toward voice-to-action agents, with recent work on speech-instructed GUI agents [22] and region-aware visual grounding [23] demonstrating that the underlying components are maturing

rapidly. Yet despite this progress across both threads, the most rigorous evaluation environment in the field — the OSWorld benchmark — shows that frontier models complete only 12.24% of real computer tasks that humans complete at 72.36% [17]. The gap between demonstration and deployable replacement remains substantial.

This rapid progress has been uneven across platforms. Windows is heavily represented in the desktop agent literature [15, 16], Android in the mobile agent literature [10, 11, 12], and the browser in the web agent literature [7, 8, 9]. Linux desktop coverage, by contrast, is conspicuously absent: existing work either addresses Linux only at the shell level [3] or treats it as one of several evaluation platforms rather than a primary target [17]. No surveyed system implements a working conversational userland[3] for the Linux desktop, despite Linux serving as the substrate for most developer and server-side workflows.

Gap / Opportunity

Three compounding gaps in the existing literature motivate the present research. First, no system in the literature has replaced the conventional desktop userland with a conversational one — existing systems automate on top of the desktop shell rather than replacing it. The UFO² system, for instance, positions itself as a "Desktop AgentOS" but operates as an automation layer atop the existing Windows shell rather than substituting for it [16]. Second, Linux desktop coverage remains absent despite Linux being the substrate for most developer and server-side workflows; the NaSh project addresses Linux but only at the shell level [3], and OSWorld includes Ubuntu only as one of three evaluation platforms rather than as a primary deployment target [17]. Third, no existing work compares a conversational interface against the conventional graphical workflow on the same tasks with the same user population [17]; user studies in this space are rare, with only VoicePilot [5] conducting a substantive human-centered evaluation of LLM-based interaction.

Notably, the accessibility-focused thread of the field exhibits the inverse limitation. Speech-to-GUI mapping research targets specific user populations — users with motor impairments, speech disorders, or cognitive disabilities — with specialized techniques such as speech-instructed GUI agents and resilient visual grounding [22, 23], but operates at the level of individual applications or interaction techniques rather than at the level of the userland itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists. This complementarity sharpens the opportunity: a general-purpose conversational userland would serve not only mainstream users but also provide the substrate that accessibility-specific configurations require.

These gaps present a clear opportunity, and a precedent for how to seize it. Android demonstrated that a new userland — a different runtime, application framework, and interaction model — can be built atop the existing Linux kernel without modifying the kernel itself, producing a platform experienced by users as an entirely distinct system. The opportunity addressed by this study is to apply the same pattern in a new direction: SynapseOS is a Linux distribution that replaces the conventional graphical userland with a conversational one, leaving the kernel and the existing Linux application ecosystem untouched. It is designed to serve as the primary user-facing interface to a personal computing environment — spanning novice, elderly, non-technical, and technical user populations — with an evaluation methodology that engages the conventional desktop workflow as its baseline rather than as its substrate.

Problem Statement

This study addresses the absence of a conversational userland for the Linux desktop — a Linux distribution replacing the desktop shell in the way Android replaced the mobile userland, rather than a modification to the kernel itself — that has been empirically evaluated against the conventional graphical workflow it would displace.

Specifically, the study seeks to answer the following questions:

1. How can a conversational userland be designed to function as the primary user-facing interface to a Linux-based personal computing environment, spanning shell operations, graphical applications, and cross-application workflows?
2. What hybrid execution architecture — combining MCP-exposed application APIs[4], Linux accessibility-tree access via AT-SPI[5], and vision-based GUI simulation as fallback — is most effective for grounding natural language intent into system actions on the Linux desktop?
3. How does SynapseOS compare to the conventional Linux graphical desktop workflow in terms of task completion time, error rate, and user satisfaction across novice and power user populations?

Objectives

The general objective of this study is to design, implement, and evaluate SynapseOS, a Linux distribution that replaces the conventional graphical userland — the desktop shell, session manager, and application launcher — with a conversational interface layer, while leaving the underlying Linux kernel and application ecosystem unchanged.

Specifically, this study aims:

1. To design a conversational userland for the Linux desktop that spans shell operations, graphical applications, and cross-application workflows under a single natural-language surface.
2. To develop a hybrid execution architecture for SynapseOS that prioritizes MCP-exposed application APIs, falls back to Linux AT-SPI accessibility-tree access, and uses vision-based GUI simulation as a last resort, balancing reliability against generality.

3. To implement SynapseOS as a Linux session manager[6] that provides a persistent conversational surface, a structured undo log, and a confirmation policy for irreversible operations, addressing the safety and recoverability gaps identified in prior work [3, 16].
4. To evaluate SynapseOS through a controlled user study comparing task completion time, error rate, and self-reported satisfaction against the conventional Linux desktop workflow, across novice and power user populations, providing the empirical comparison absent from the existing literature [17].

Significance of the Study

The findings of this study are expected to benefit the following groups:

1. **Novice and non-technical computer users.** The conventional graphical desktop imposes a steep learning curve rooted in its icon-and-menu paradigm. A conversational userland that accepts natural language intent lowers the threshold for productive computer use, particularly for users with limited prior computing experience, including elderly users encountering desktop computing late or infrequently.
2. **Developers and power users on Linux.** A conversational userland reduces the cognitive load of switching between shell commands, graphical application controls, and cross-application workflows by exposing system operations through a unified natural-language interface.
3. **Users with motor accessibility needs.** Although this population is scoped out of the initial evaluation for ethical and recruitment reasons, SynapseOS is positioned to serve as the userland-level substrate on which accessibility-specific configurations can operate. Prior work has established design guidelines for LLM-based speech interfaces for users with motor impairments [5] and has demonstrated specialized techniques — speech-instructed GUI

agents and resilient visual grounding — at the application and interaction-technique level [22, 23]. These techniques presuppose a conversational system surface that does not yet exist at the userland level; SynapseOS provides that surface, making accessibility-specific extensions a natural and well-grounded future direction rather than a speculative one.

4. **The research community.** The controlled comparative evaluation methodology this study introduces addresses a recognized gap in the literature, where conversational interfaces are rarely compared against conventional graphical workflows on the same tasks and population [17]. The evaluation design provides a replicable methodology for future work in this space.

1.1 Scope and Limitations of the Study

This study covers the design, implementation, and evaluation of SynapseOS as a conversational userland for the Linux desktop. SynapseOS is implemented as a Linux distribution that replaces the conventional userland — the desktop shell, session manager, and application launcher — with a conversational layer, analogous in scope to how Android replaced the conventional userland atop the Linux kernel for mobile devices. The Linux kernel itself, and the existing ecosystem of Linux applications, are not modified by this study; SynapseOS interacts with applications through the action mechanisms described in Section VI of the preceding survey (MCP APIs, AT-SPI, and vision-based fallback) rather than by rewriting them. The subject matter spans natural language processing, LLM-based reasoning, graphical user interface automation, Linux system integration, and human-computer interaction.

The study examines cross-application desktop workflows on Linux, including shell operations, file management, and application-level tasks representable in the OSWorld benchmark task suite [17]. The population studied consists of novice

computer users and power users recruited for a controlled user study. The study is conducted in a Linux desktop environment and is bounded by the duration of the thesis project.

The following limitations are acknowledged. First, the choice of underlying multimodal LLM is not yet committed; the decision between a hosted frontier model and a locally-hosted open-weights model[7] involves tradeoffs between latency, privacy, and capability that will affect results. Second, users with motor accessibility needs — a highly relevant population — are excluded from the initial evaluation due to ethics-review and recruitment constraints; accessibility-specific configurations such as speech-instructed agent operation for atypical speech patterns [22] are likewise out of scope for the initial implementation but represent a natural extension of the SynapseOS substrate. Third, SynapseOS targets the Linux userland specifically, and the findings may not generalize directly to Windows or macOS desktop environments. Fourth, the safety confirmation policy for irreversible system operations is defined in principle but requires empirical calibration during implementation. Fifth, evaluation is bounded by the task coverage of the OSWorld benchmark and the custom task suite developed for this study; tasks outside this coverage are not evaluated.

Notes

1. Foundation models are large neural networks trained on broad, general-purpose data and adaptable to many tasks. Multimodal variants accept multiple input types — typically text and images — enabling the model to interpret visual interfaces and screenshots alongside natural language.
2. Computer-using agents are AI systems that interact with a computer's graphical environment — clicking, typing, reading the screen — to complete tasks on behalf of a user. Unlike shell-level automation, they operate through the same visual interface a human would use.
3. The userland (or user space) is the layer of software that runs on top of the operating system kernel and constitutes the user-facing computing environment: the desktop shell, window manager, session manager, application launcher, and the applications themselves. The kernel manages hardware and process scheduling; the userland defines what the user sees and interacts with.

4. Model Context Protocol (MCP): an open standard for connecting AI models to external tools and application APIs through a structured interface. Applications that implement MCP expose their functions as callable tools, enabling reliable structured interaction without screen-based interpretation.
5. Assistive Technology Service Provider Interface (AT-SPI): a Linux accessibility protocol that exposes the UI elements of graphical applications — buttons, text fields, menus — as a queryable tree structure, enabling programmatic interaction without modifying application source code.
6. A Linux session manager is the program that starts after user login and establishes the graphical environment — launching the window manager, desktop shell, and background services. SynapseOS replaces this component with a conversational interface, making it the first program a user encounters after login.
7. Open-weights models make their trained parameters publicly available for local download and use. Users run inference on their own hardware without routing queries to an external service provider, enabling offline operation and preventing user data from being transmitted to third parties.

References

[1] Wilensky, R., Chin, D. N., Luria, M., Martin, J., Mayfield, J., & Wu, D. (1988). The Berkeley Unix Consultant Project. *Computational Linguistics*, 14(4), 35–84. https://aclanthology.org/J88-4003/

[2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. https://arxiv.org/abs/2502.06858

[3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. https://arxiv.org/abs/2506.13028

[5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. https://doi.org/10.1145/3654777.3676401

[7] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2306.06070

[8] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of ICLR 2024*. https://arxiv.org/abs/2307.13854

[9] Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. (2024). GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of ICML 2024*. https://arxiv.org/abs/2401.01614

[10] Rawles, C., Li, A., Rodriguez, D., Riva, O., & Lillicrap, T. (2023). Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. https://arxiv.org/abs/2307.10088

[11] Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., & Yu, G. (2023). AppAgent: Multimodal Agents as Smartphone Users. *arXiv:2312.13771*. https://arxiv.org/abs/2312.13771

[12] Wang, J., Xu, H., Ye, J., Yan, M., Shen, W., Zhang, J., Huang, F., & Sang, J. (2024). Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv:2401.16158*. https://arxiv.org/abs/2401.16158

[15] Hui, T., Zhang, Y., Jiang, B., Sun, J., Cheng, F., Lin, X., & Yang, Y. (2025). WinClick: GUI Grounding with Multimodal Large Language Models. *arXiv:2503.04730*. https://arxiv.org/abs/2503.04730

[16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. *arXiv:2504.14603*. https://arxiv.org/abs/2504.14603

[17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. https://arxiv.org/abs/2404.07972

[18] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Large Language Model-Brained GUI Agents: A Survey. *arXiv:2411.18279*. https://arxiv.org/abs/2411.18279

[22] Han, W., Zeng, Z., Huang, J., Jiang, S., et al. (2025). GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. *arXiv:2506.11127*. https://arxiv.org/abs/2506.11127

[23] Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., & Ghadar, S. (2025). R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 9669–9685. https://doi.org/10.18653/v1/2025.findings-acl.501